



Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

2026-02-10

ChanWoo Kim

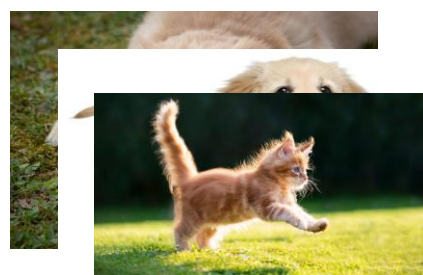
MINDSLab

- Training Deep Neural Networks
- Motivation for Batch Normalization
- Method: Batch Normalization
- Why BN works
- Experiments
- Conclusion & Future works

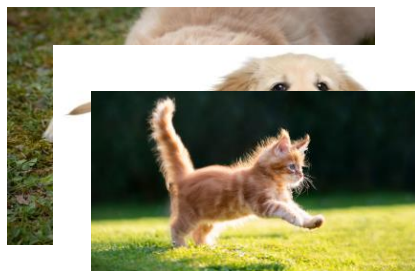
Training Deep Neural Networks

- **What is Internal Covariate Shift?**

- Change in distribution of a layer's inputs during training as earlier layer parameters are updated

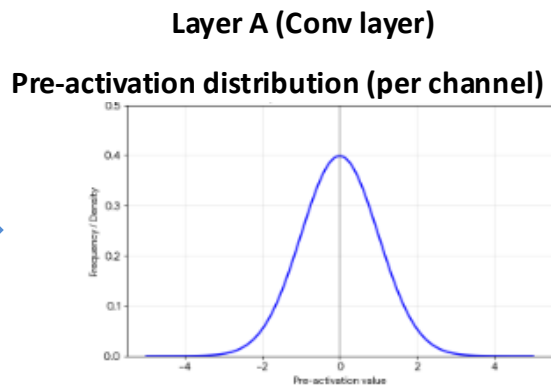


Mini-batch 0 (input images)



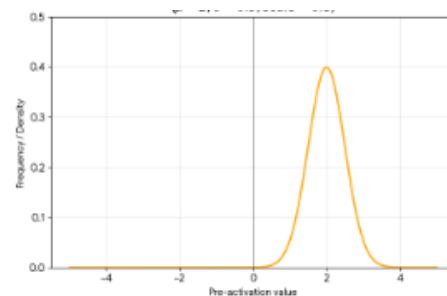
Mini-batch 0 (input images)

input
→



Before parameter update

input
→



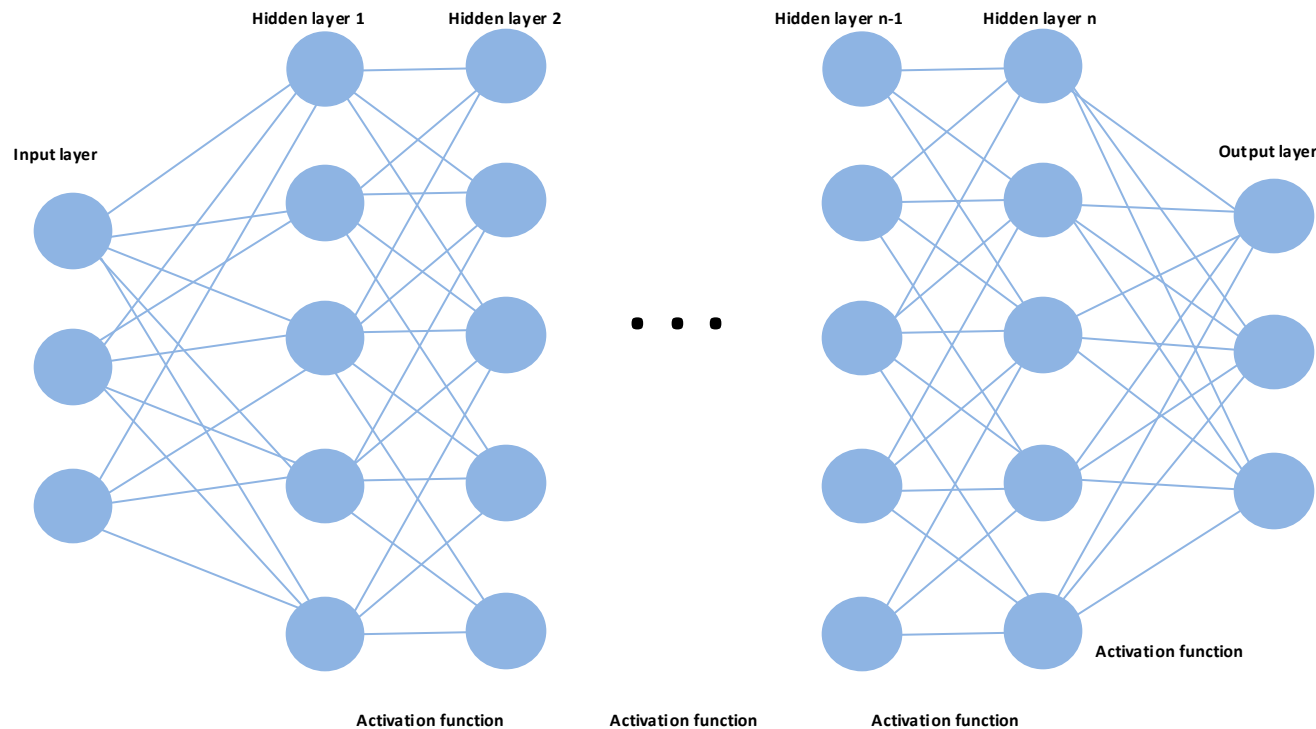
After parameter updates



Layer (each channel) has to adapt to
new input distribution
every mini-batch

Training Deep Neural Networks

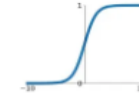
- **Internal Covariate Shift with saturating non-linearities?**
 - Activation functions that have area which derivatives are near zero



Activation Functions

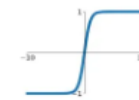
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Small differences get amplified

- Initialization problem
- Gradient vanishing/exploding

Training Deep Neural Networks

- Why does small differences get amplified?

$$\ell = \mathcal{L}(F_k(F_{k-1}(\dots F_2(F_1(x, \Theta_1), \Theta_2) \dots), \Theta_{k-1}), \Theta_k)$$

$$y = W_k \cdot W_{k-1} \cdot \dots \cdot W_2 \cdot (W_1 x)$$

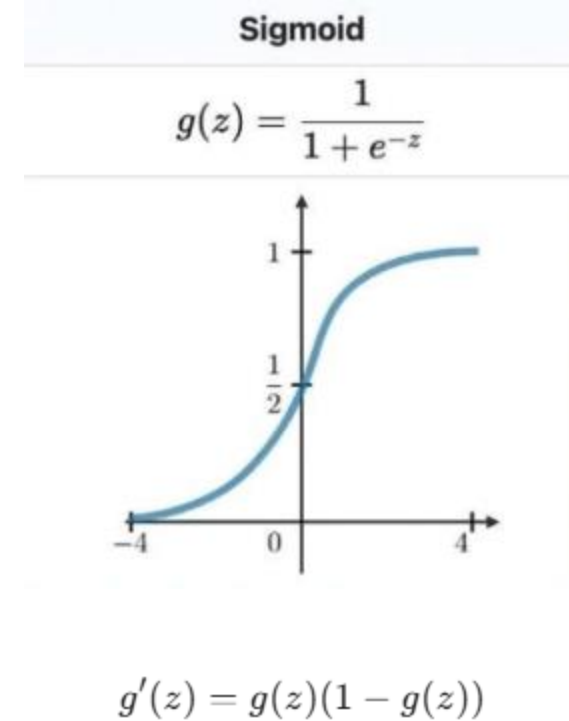
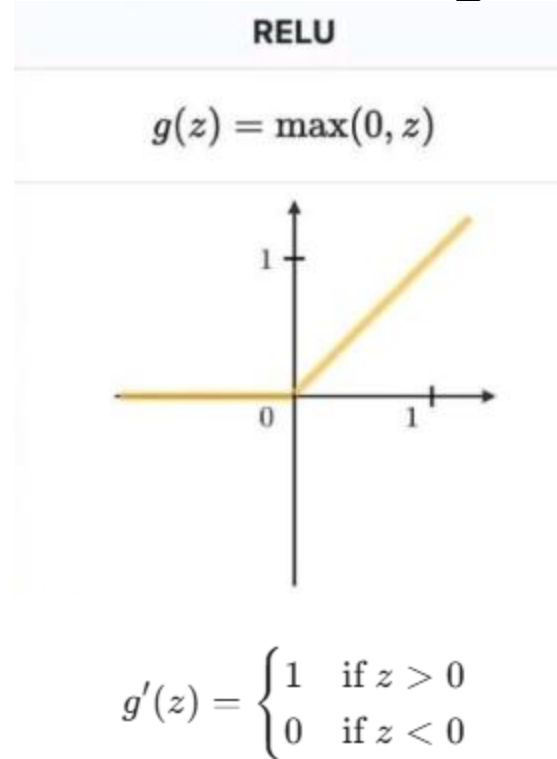
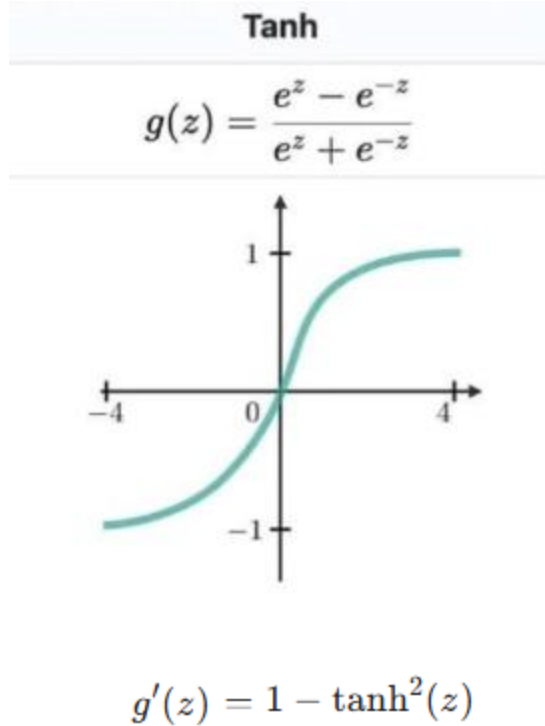
$$y_{new} = W_k \cdot \dots \cdot W_2 \cdot (W_1 + \Delta W_1)x$$

$$y_{new} = (W_k \cdot \dots \cdot W_1 x) + (W_k \cdot \dots \cdot W_2) \cdot (\Delta W_1 x)$$

$$y_{new} = (W_k \cdot \dots \cdot W_1 x) + (W_k \cdot \dots \cdot W_2) \cdot (\Delta W_1 x)$$

Training Deep Neural Networks

- Internal Covariate Shift with saturating non-linearities



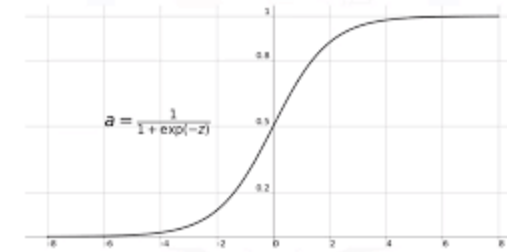
➡ Small differences in values get amplified regardless of the activation function

Training Deep Neural Networks

- Example: sigmoid function in two layers

$$\sigma(a) = \frac{1}{1 + e^{-a}} \quad \sigma'(a) = \sigma(a) (1 - \sigma(a))$$

Sigmoid Function



→ $\frac{dL}{dx} = \frac{dL}{dh_2} \cdot \frac{dh_2}{da_2} \cdot \frac{da_2}{dh_1} \cdot \frac{dh_1}{da_1} \cdot \frac{da_1}{dx}$

$$a_1 = w_1x + b_1, \quad h_1 = \sigma(a_1)$$

$$a_2 = w_2h_1 + b_2, \quad h_2 = \sigma(a_2)$$

→ $\frac{dL}{dx} = \frac{dL}{dh_2} \cdot \sigma'(a_2) \cdot w_2 \cdot \sigma'(a_1) \cdot w_1$

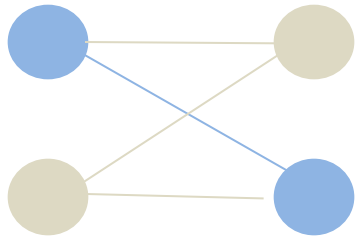
$$\sigma'(a_1)\sigma'(a_2) \leq (0.25)^2$$

Activations shift to saturated regions, the derivative becomes almost zero

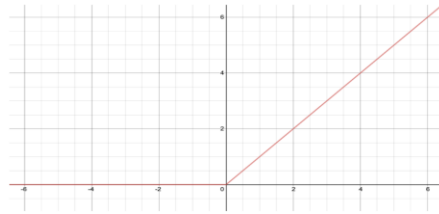
Motivation for Batch Normalization

- Conventional approaches

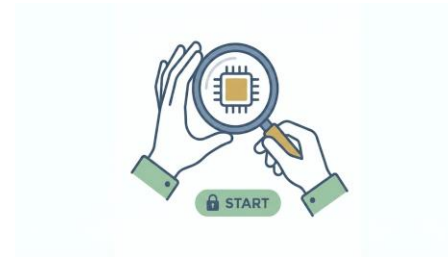
Dropout



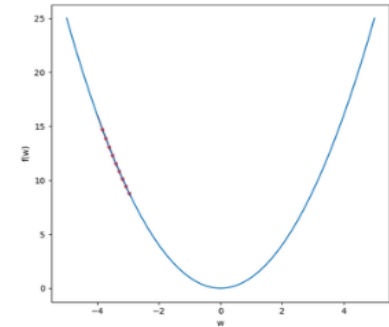
ReLU



Careful initialization



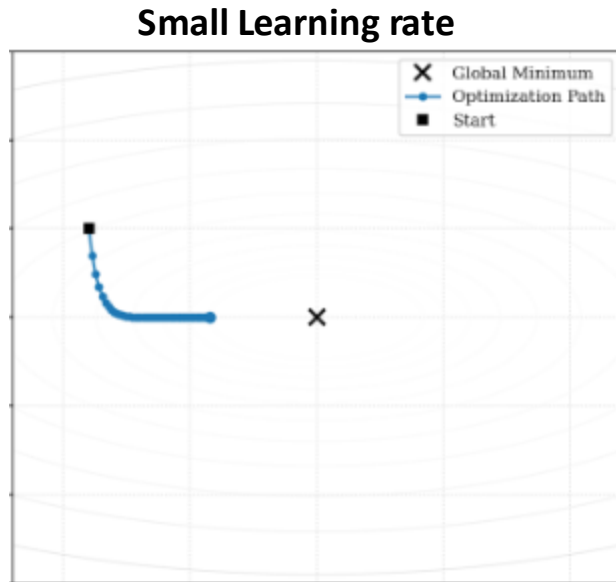
Small learning rate



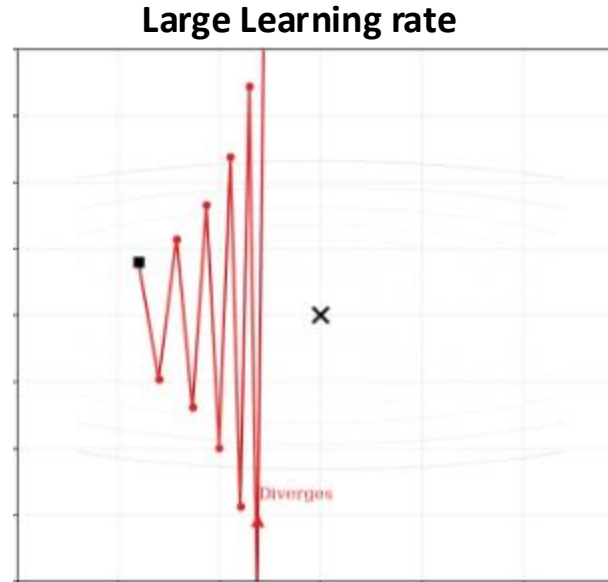
Cannot solve the problem fully

Motivation for Batch Normalization

- Conventional approaches: Small learning rate



Converges eventually



Diverge



Low learning rate prevent updates
amplifying across deep layers



But it is hard to converge with small learning rate

Method: Batch Normalization



Training

Input: Values of x over a mini-batch: $B =$

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1} x_i \quad // \text{ mini-batch mean}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1} (x_i - \mu_B)^2 \quad // \text{ mini-batch variance}$$

$$\tilde{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \tilde{x}_i + \beta = \text{BN}_{\gamma, \beta}(x) \quad // \text{ scale and shift}$$

Inference

Input: Activation x to be normalized (inference)

Moving averages: μ_{run} (running mean), σ_{run}^2 (running variance)

Learned parameters: γ, β

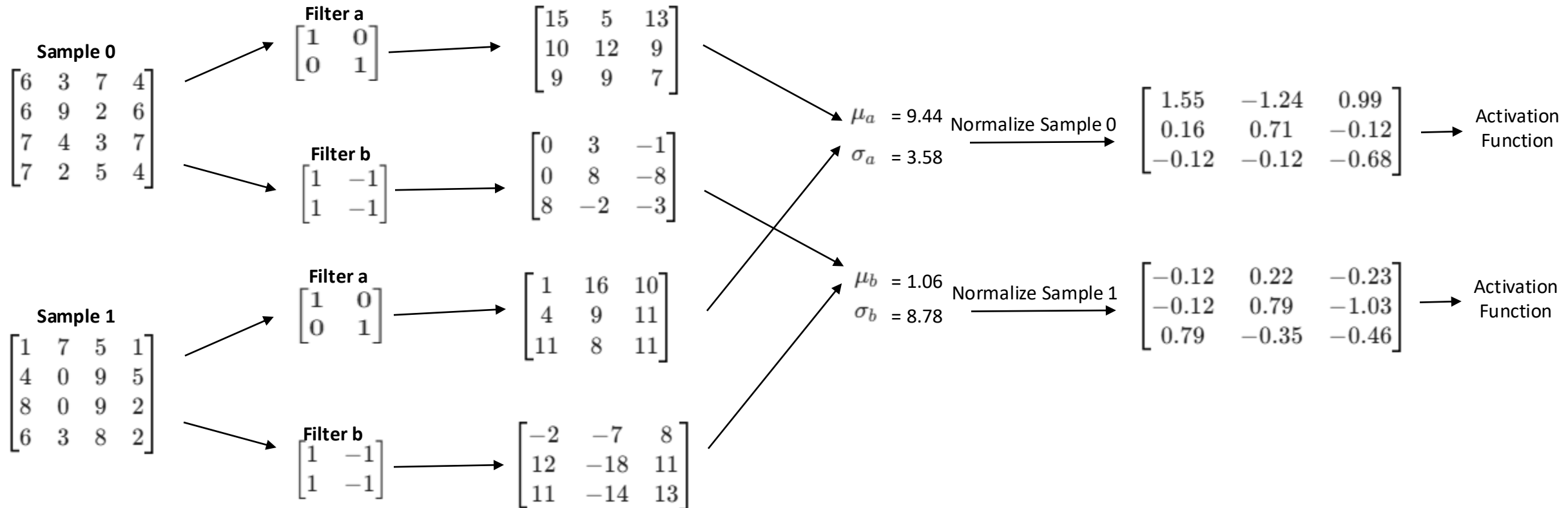
Output: $y = \text{BN}_{\gamma, \beta}(x)$

$$\hat{x} \leftarrow \frac{x - \mu_{\text{run}}}{\sqrt{\sigma_{\text{run}}^2 + \epsilon}} \quad // \text{ normalize using running mean/var}$$

$$y \leftarrow \gamma \hat{x} + \beta \quad // \text{ scale and shift}$$

Method: Batch Normalization

■ Example: Training 4x4 image (Batch size = 2)



Method: Batch Normalization

- Example: Inferencing 4x4 image input

Moving average/variance during training

$$\mu_{run,n} = \frac{1}{n} \sum_{i=1}^n \mu_{B,i} \quad \sigma_{run,n}^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{m}{m-1} \cdot \sigma_{B,i}^2 \right)$$

Sample 0

$$\begin{bmatrix} 6 & 3 & 7 & 4 \\ 6 & 9 & 2 & 6 \\ 7 & 4 & 3 & 7 \\ 7 & 2 & 5 & 4 \end{bmatrix}$$

Filter a

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 15 & 5 & 13 \\ 10 & 12 & 9 \\ 9 & 9 & 7 \end{bmatrix}$$

$$\begin{aligned} \mu_{run} &= 3 \\ \sigma_{run} &= 2 \end{aligned}$$

$$\hat{x} = \begin{bmatrix} 6 & 1 & 5 \\ 3.5 & 4.5 & 3 \\ 3 & 3 & 2 \end{bmatrix}$$

$$y = \begin{bmatrix} 6.0 & 1.0 & 5.0 \\ 3.5 & 4.5 & 3.0 \\ 3.0 & 3.0 & 2.0 \end{bmatrix}$$

$$\hat{x} \leftarrow \frac{x - \mu_{run}}{\sqrt{\sigma_{run}^2 + \epsilon}}$$

$$y \leftarrow \gamma \hat{x} + \beta$$

$\gamma = 1, \beta = 0$

Why BN works

- **Whitening: parameters converge faster when inputs are whitened**
 - Zero means, unit variances
- **Learnable scale & shift parameters**

Zero means, unit variances



Since scale of input is normalized, update step size is also consistent

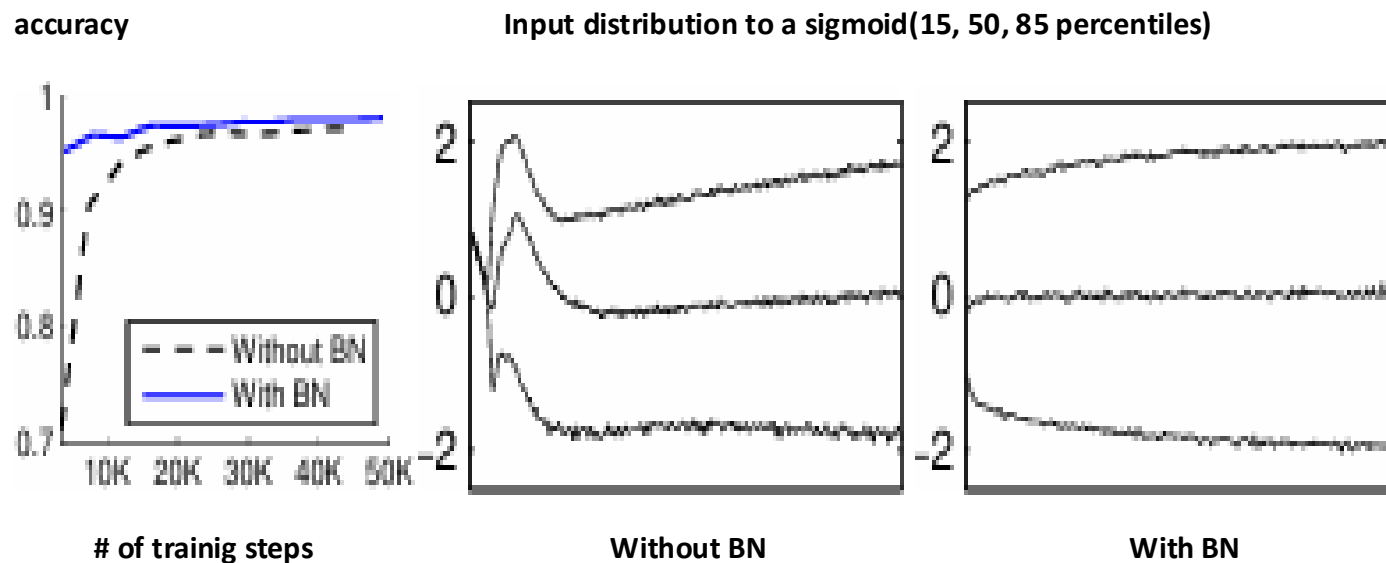
Learnable parameters

$$y \leftarrow \gamma \hat{x} + \beta$$

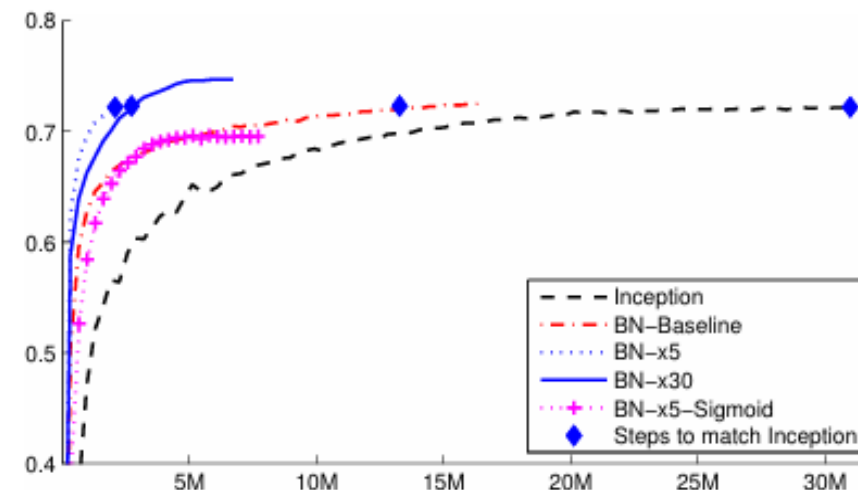
- Restoring representation power
- Optimizing distribution for efficient training

EXPERIMENTS

■ Difference in accuracy



Single crop validation accuracy



of training steps

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

EXPERIMENTS

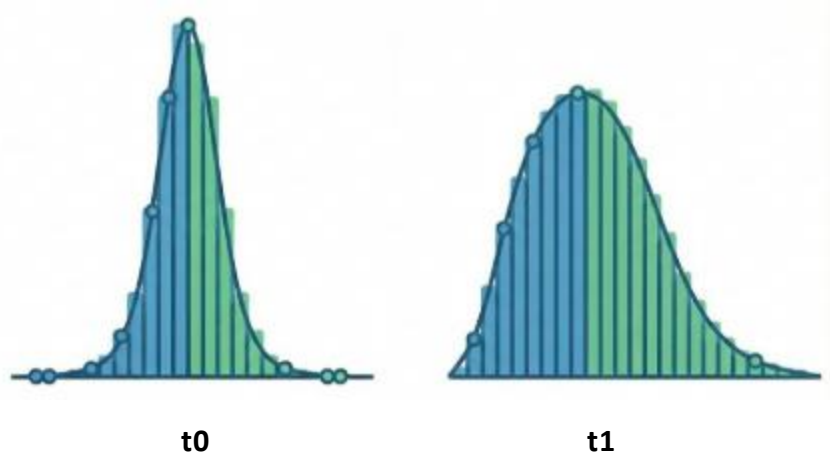
- Comparison between sota in image data

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	up to 512	-	-	-	5.98%
MSRA multicrop	up to 480	-	-	-	5.71%
MSRA ensemble	up to 480	-	-	-	4.94%*
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	4.82%*

Future Works

- BN is dependent on batch size
- Inappropriate for sequence data
- Not suitable for Online learning environments

BN is not appropriate for sequence datas



BN cannot be used in online learning environments



Conclusion



- **Significant Training SpeedUp**
 - Allows networks to reach sota accuracy with far fewer training steps
- **Higher Learning rates**
 - It can be used without risk of divergence or vanishing gradients
- **Support for Saturating Non-linearities**
 - Prevents models from getting stuck in the saturated regimes
- **Performance Breakthroughs**
 - Achieved lower test error in imageNet classification task

Q&A